UniEVANGÉLICA

Internet das Coisas

Documento de requisitos · v2.0

Documento de Requisitos Plataforma IoT Integrada

Plataforma IoT básica para integrar projetos urbanos e rurais, independente de linguagem e de cenário, em camadas e com padrões de mercado.

Versão 2.0 · outubro de 2026

SUMÁRIO

Conteúdo

Introdução . . . 3

1.1 Propósito . . . 3

1.2 Escopo . . . 3

1.3 Público-alvo . . . 3

1.4 Convenções . . . 4

1.5 Histórico de revisões . . . 4

Descrição geral . . . 5

2.1 O problema . . . 5

2.2 Cenários e domínios de referência . . . 5

2.3 O que muda entre a cidade e o campo . . . 6

2.4 Atores . . . 6

2.5 Premissas e restrições . . . 6

Arquitetura de referência em camadas . . . 8

3.1 Visão em camadas . . . 8

3.2 Responsabilidades . . . 9

3.3 Visão de implantação: central e borda . . . 9

3.4 Correspondência com plataformas de mercado . . . 10

Integração independente de linguagem e de cenário . . . 11

4.1 Princípio: integrar por contrato, não por biblioteca . . . 11

4.2 Os três níveis de integração . . . 11

4.3 Hierarquia de locais . . . 12

4.4 Mensagem canônica . . . 12

4.5 Convenção de tópicos MQTT . . . 13

4.6 API REST (resumo) . . . 13

Requisitos funcionais . . . 14

5.1 Cenários e catálogo (CEN) . . . 14

5.2 Conectividade e ingestão (ING) . . . 14

5.3 Borda e conectividade limitada (EDG) . . . 15

5.4 Gestão de dispositivos (DIS) . . . 16

5.5 Integração de projetos existentes (INT) . . . 16

5.6 Processamento e regras (PRO) . . . 17
5.7 Dados (DAD) . . . 17
5.8 Aplicação e serviços (APP) . . . 18
5.9 Segurança e acesso (SEG) . . . 18
5.10 Observabilidade e administração (OBS) . . . 19
Requisitos não funcionais . . . 20
Padrões de projeto e boas práticas . . . 21
7.1 Arquitetura de cada serviço: hexagonal . . . 21
7.2 Padrões adotados . . . 21
7.3 Boas práticas de programação e engenharia . . . 22
Casos de uso . . . 23
Tecnologias de referência e roteiro . . . 25
9.1 Tecnologias de referência . . . 25
9.2 Roteiro de entrega . . . 25
9.3 Critérios de aceite do MVP . . . 26
Glossário e referências . . . 27
10.1 Glossário . . . 27
Referências . . . 27

SEÇÃO 1

Introdução

Propósito, escopo, público, convenções e histórico deste documento.

1.1 Propósito

Este documento especifica os requisitos da **Plataforma IoT Integrada**, uma plataforma IoT básica que permite integrar, num só lugar, projetos de IoT que hoje funcionam isolados, **em qualquer cenário**: cidades inteligentes, fazendas, cooperativas, agroindústrias ou outros que surgirem. A plataforma deve ser independente de **linguagem de programação** (qualquer projeto se integra usando apenas protocolos e contratos abertos) e independente de **cenário** (o núcleo não contém nenhum conceito urbano ou rural fixo; os cenários são configurados).

1.2 Escopo

A plataforma recebe dados de dispositivos e sistemas, gerencia os dispositivos, processa regras e alarmes, armazena histórico e estado, e disponibiliza tudo por APIs e painéis. Para locais com conectividade limitada, como propriedades rurais, oferece um **nó de borda** que funciona desconectado e sincroniza depois. A arquitetura segue as camadas das principais plataformas de mercado (AWS IoT, Azure IoT, ThingsBoard e FIWARE), numa versão enxuta, de código aberto e implantável em qualquer infraestrutura.

Dentro do escopo	Fora do escopo
Ingestão por MQTT, HTTP, CoAP e lotes; integração com servidores LoRaWAN	Projeto e fabricação de dispositivos e gateways
Catálogo configurável de domínios, perfis de dispositivo e locais	Implantação da infraestrutura de rede (antenas, LoRaWAN, enlaces de satélite)
Nó de borda com fila local, regras locais e sincronização	Modelos avançados de IA e gêmeos digitais 3D
Cadastro, credenciais, gêmeo, comandos e regras; séries temporais e contexto NGSI-LD	Sistemas de gestão agrícola ou urbana completos (ERP, gestão de safra, tributos)
APIs, webhooks, painéis, notificações e dados abertos	Aplicativo móvel nativo (fase futura; a interface web deve ser responsiva)

1.3 Público-alvo

- **Equipes de projeto e integradores**, urbanos ou rurais, que conectarão soluções existentes à plataforma.
- **Equipe de desenvolvimento** da plataforma, que implementará os módulos.
- **Gestores** de prefeituras, cooperativas e propriedades rurais, que validarão escopo e prioridades.
- **Professores e estudantes**, que usarão o documento como referência de arquitetura IoT.

1.4 Convenções

Elemento	Convenção
Identificador	RF-XXX-nn (requisito funcional do módulo XXX) e RNF-nn (requisito não funcional).
Prioridade	**Essencial**: obrigatório no MVP. **Importante**: fase 2. **Desejável**: fase 3 ou conforme disponibilidade.
Verbos	“Deve” indica obrigação; “pode” indica opção. Cada requisito tem um critério de aceitação verificável.
Base normativa	Estrutura inspirada na ISO/IEC/IEEE 29148 (engenharia de requisitos) e na arquitetura de referência ISO/IEC 30141 (IoT).

1.5 Histórico de revisões

Versão	Data	Alteração
1.0	19/10/2026	Versão inicial, voltada a projetos de cidades inteligentes (“Plataforma IoT Urbana”).
2.0	19/10/2026	Cenário flexível: projetos urbanos e rurais. Novos módulos Cenários e catálogo (CEN) e Borda e conectividade limitada (EDG); dados atrasados, lotes, locais e áreas; casos de uso rurais.

SEÇÃO 2

Descrição geral

O problema, os cenários a atender, o que muda entre a cidade e o campo, os atores e as premissas.

2.1 O problema

Projetos de IoT costumam nascer isolados: cada um tem seu sensor, seu servidor, seu banco e seu painel. Na cidade, é o desafio de **interoperabilidade** visto na aula de Cidades Inteligentes. No campo, o mesmo problema aparece numa única fazenda: a estação meteorológica de um fabricante, os sensores de solo de outro, a telemetria das máquinas num portal próprio e o controle dos silos numa planilha. O resultado é o mesmo nos dois casos: dados que não conversam, credenciais espalhadas e nenhuma visão integrada.

A plataforma oferece uma base comum, sem obrigar os projetos a reescrever o que já funciona, e **sem ser desenhada para um único cenário**: o que diferencia uma praça de um talhão é configuração, não código.

2.2 Cenários e domínios de referência

Os domínios abaixo são **exemplos de partida**. O catálogo é configurável (RF-CEN-01): novos domínios podem ser criados sem alterar a plataforma.

Cenário	Domínio	Exemplos de projetos	Dados típicos
Urbano	Mobilidade	Semáforos, estacionamento, contagem de veículos	Fluxo, vagas livres
Urbano	Iluminação pública	Telegestão e dimerização de luminárias	Estado, potência, falhas
Urbano	Resíduos	Lixeiras com sensor de nível, rotas de coleta	Nível de enchimento
Urbano	Meio ambiente	Qualidade do ar e ruído, irrigação de praças	PM2.5, CO2, dB
Urbano	Energia	Medidores em prédios públicos	Consumo, geração
Rural	Agricultura	Sensores de solo, irrigação por talhão	Umidade e temperatura do solo
Rural	Clima	Estações meteorológicas da propriedade	Chuva, vento, temperatura
Rural	Armazenagem	Termometria de silos e armazéns	Temperatura e umidade dos grãos
Rural	Pecuária	Brincos e colares com GPS, bebedouros	Posição, consumo de água
Rural	Máquinas	Telemetria de tratores e colheitadeiras	Horas, combustível, posição
Ambos	Recursos hídricos	Reservatórios, poços, rios	Nível, vazão

2.3 O que muda entre a cidade e o campo

O núcleo é o mesmo; as condições de operação, não. A coluna da direita mostra como a plataforma absorve cada diferença sem ter um “modo rural” no código:

Aspecto	Cidade	Campo	Resposta da plataforma
Conectividade	Fibra, 4G/5G, LoRaWAN denso	LoRaWAN de longo alcance, celular fraco, satélite	Nó de borda com fila; dados atrasados e lotes
Energia	Rede elétrica disponível	Bateria e painel solar	Cargas compactas (CBOR), envios espaçados
Localização	Endereços e pontos	Talhões, pastos e animais em movimento	Locais, áreas e geometrias GeoJSON
Densidade	Muitos dispositivos próximos	Poucos dispositivos em áreas grandes	Perfis com intervalo esperado por tipo
Operação	Centro de operações	Produtor no celular, conexão eventual	Interface responsiva e notificação por SMS
Dados pessoais	Cidadãos, placas de veículos	Produtor, propriedade, produção	LGPD nos dois casos

2.4 Atores

Ator	Papel	Exemplos
Administrador da plataforma	Opera e configura a plataforma como um todo	Equipe de TI da instituição
Gestor do inquilino	Responsável por um projeto ou organização	Secretaria municipal, cooperativa, fazenda
Integrador	Desenvolvedor que conecta um projeto existente	Equipe do projeto, fornecedor
Operador	Acompanha painéis e alarmes	Centro de operações, produtor, agrônomo
Dispositivo, sistema ou nó de borda	Publica dados e recebe comandos	Sensor, gateway, sistema legado
Cidadão / pesquisador	Consome dados abertos	Universidade, imprensa, população

2.5 Premissas e restrições

- Todo projeto integrado envia dados por rede IP, diretamente, por um gateway ou por um **nó de borda** que guarda e reenvia quando não há conexão contínua.
- A plataforma usa **componentes de código aberto** e não depende de serviço proprietário de um único provedor de nuvem no seu núcleo.
- Toda interface externa usa **protocolos abertos** (MQTT, HTTP, CoAP) e **contratos públicos** (OpenAPI, AsyncAPI, JSON Schema). Nenhuma biblioteca proprietária pode ser obrigatória para integrar.
- O código do núcleo não contém domínios, tipos de dispositivo ou regras específicos de um cenário.
- O tratamento de dados pessoais segue a **LGPD** (Lei nº 13.709/2018).

- O MVP funciona num único servidor central, mais um nó de borda por local com conexão limitada.

SEÇÃO 3

Arquitetura de referência em camadas

As camadas da plataforma, a visão de implantação central e de borda, e a correspondência com as plataformas de mercado.

3.1 Visão em camadas

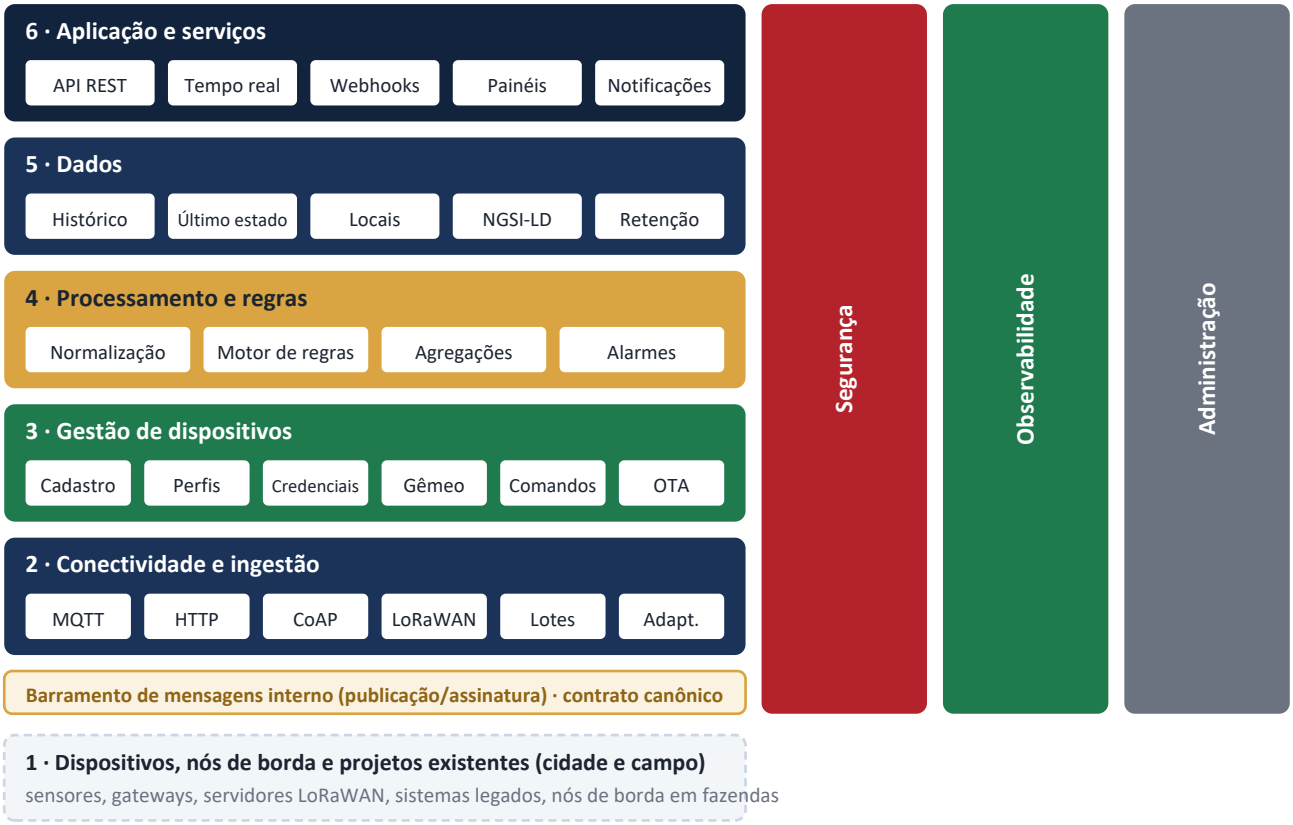


Figura 3.1 · Arquitetura de referência em camadas. As camadas 2 a 6 compõem a plataforma central; as três colunas à direita atravessam todas elas. Nenhuma camada conhece o cenário: domínios, perfis e locais são configuração.

Cada camada só conversa com a camada vizinha ou com o **barramento interno**, por meio de contratos. Assim, é possível trocar a implementação de uma camada (por exemplo, o banco de séries temporais) sem alterar as demais.

3.2 Responsabilidades

Camada	Responsabilidade	Não faz
1 · Dispositivos e borda	Medir, atuar e enviar dados; nos nós de borda, guardar, decidir localmente e sincronizar.	—
2 · Conectividade e ingestão	Receber mensagens e lotes por vários protocolos, autenticar, validar e converter ao formato canônico.	Regras de negócio
3 · Gestão de dispositivos	Cadastro, perfis, credenciais, grupos, gêmeo, comandos e OTA.	Armazenar telemetria
4 · Processamento e regras	Normalizar, agregar, avaliar regras e gerar alarmes e ações.	Falar protocolo de dispositivo
5 · Dados	Guardar séries temporais, último estado, locais e áreas, contexto e metadados.	Decidir ações
6 · Aplicação e serviços	Expor APIs, tempo real, webhooks, painéis e notificações.	Acessar o banco sem passar pelos serviços
Transversais	Segurança (identidade e acesso), observabilidade e administração.	—

3.3 Visão de implantação: central e borda

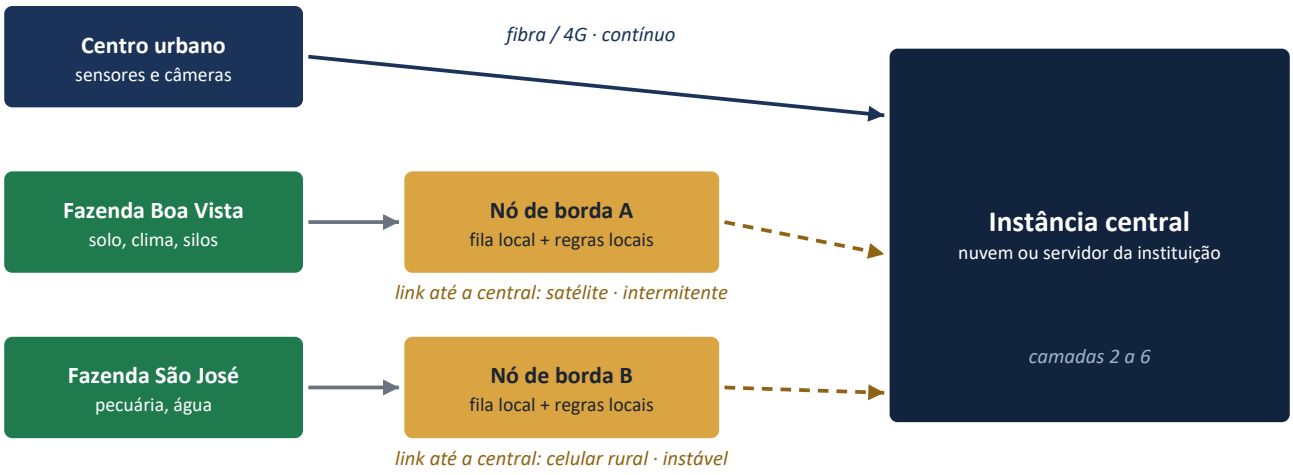


Figura 3.2 · Visão de implantação. Locais com conexão contínua falam direto com a instância central; locais com conexão limitada usam um nó de borda que guarda, decide localmente e sincroniza quando o link volta.

O **nó de borda** é uma versão reduzida da plataforma (ingestão, fila persistente e motor de regras local), instalada na propriedade em hardware modesto. Ele é opcional: um bairro com fibra não precisa dele, e uma fazenda com link via satélite que cai à noite depende dele. Como fala o mesmo contrato da instância central, os dispositivos não percebem a diferença.

3.4 Correspondência com plataformas de mercado

As camadas reproduzem o que as plataformas mais adotadas têm em comum, inclusive a borda:

Camada	AWS IoT	Azure IoT	ThingsBoard	FIWARE
Conectividade e ingestão	IoT Core (broker)	IoT Hub	Transportes MQTT/HTTP/CoAP	IoT Agents
Gestão de dispositivos	Registry, Device Shadow, Jobs	Device twin, DPS	Devices e perfis	Provisionamento nos IoT Agents
Processamento e regras	Rules Engine	Roteamento, Stream Analytics	Rule Engine	Perseo (CEP)
Dados	S3 e bancos de séries	Data Explorer	PostgreSQL / Timescale	Orion-LD, QuantumLeap
Aplicação	APIs, QuickSight	APIs, Power BI	Dashboards, REST	APIs NGSI-LD, Wirecloud
Borda	IoT Greengrass	IoT Edge	ThingsBoard Edge	Componentes FIWARE locais
Segurança	IAM, X.509	Entra ID, X.509	Tenants, RBAC	Keyrock, Wilma

SEÇÃO 4

Integração independente de linguagem e de cenário

Contrato canônico, locais e áreas, tópicos, API e os três níveis de integração.

4.1 Princípio: integrar por contrato, não por biblioteca

A independência de linguagem vem de uma decisão de projeto: **a plataforma não fornece uma biblioteca obrigatória**. Ela publica contratos, e qualquer linguagem capaz de falar MQTT ou HTTP e montar um JSON consegue se integrar, seja o firmware de um sensor de solo, seja o servidor de uma prefeitura.

Contrato	Formato	O que descreve
Mensagem canônica	JSON Schema (versionado)	Telemetria, eventos, estado, comandos e lotes.
Interface assíncrona	AsyncAPI 3	Tópicos MQTT, direção das mensagens e esquemas de cada tópico.
Interface síncrona	OpenAPI 3	Endpoints REST, parâmetros, respostas e códigos de erro.
Locais e geometrias	GeoJSON (RFC 7946)	Pontos, áreas (polígonos) e trajetórias.
Dados de contexto	NGSI-LD + Smart Data Models	Entidades padronizadas dos domínios Smart Cities e AgriFood.

4.2 Os três níveis de integração

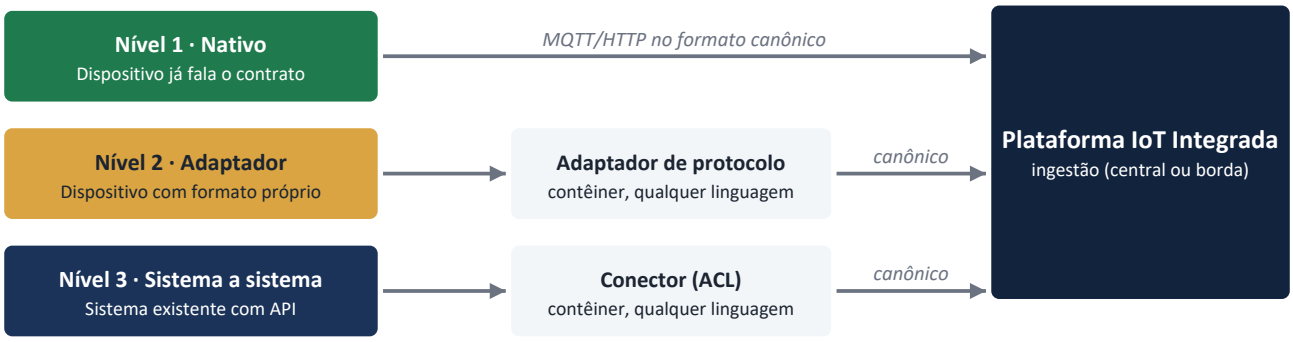


Figura 4.1 · Os três níveis de integração. Valem igualmente na cidade e no campo, e o destino pode ser a instância central ou um nó de borda.

Nível	Quando usar	O que o projeto precisa fazer
1 · Nativo	Projetos novos ou firmwares que podem ser alterados.	Publicar no tópico e no formato canônicos.
2 · Adaptador	Dispositivos com formato próprio que não podem mudar.	Fornecer (ou usar) um adaptador que traduz o formato para o canônico.
3 · Sistema a sistema	Projetos com servidor e API próprios (ex.: portal de telemetria de máquinas).	Expor a API ou enviar webhooks; um conector faz a tradução (camada anticorrupção).

4.3 Hierarquia de locais

Nível	Na cidade	No campo
Inquilino	Secretaria do Meio Ambiente	Cooperativa ou grupo agrícola
Local (site)	Bairro Centro	Fazenda Boa Vista
Área	Praça Bom Jesus	Talhão 07, pasto 3, silo 2
Dispositivo	Estação de qualidade do ar	Sensor de solo, colar de GPS, termometria

A mesma estrutura serve aos dois cenários. Locais e áreas têm geometria GeoJSON (ponto ou polígono), o que permite perguntas como “quais sensores estão dentro do talhão 07?” ou “quais lixeiras ficam neste bairro?” sem regras específicas de cenário.

4.4 Mensagem canônica

telemetria no formato canônico (versão 1.0) · exemplo rural vindo de um nó de borda

```
{
  "versao": "1.0",
  "id_mensagem": "b6d1e0c4-5f7a-4c2e-9d13-8a2f61c0e9b7",
  "dispositivo": "solo-talhao07-02",
  "tipo": "telemetria",
  "timestamp": "2026-10-19T05:10:00Z",
  "via": "borda-fazenda-boa-vista",
  "local": {
    "site": "fazenda-boa-vista", "area": "talhao-07",
    "geo": { "type": "Point", "coordinates": [-49.1012, -16.4123] }
  },
  "medidas": [
    { "grandeza": "umidade_solo",     "valor": 23.5, "unidade": "%" },
    { "grandeza": "temperatura_solo", "valor": 21.8, "unidade": "Cel" }
  ]
}
```

Campo	Regra
versao	Versão do contrato; permite evoluir o formato sem quebrar quem usa a versão anterior.
id_mensagem	UUID gerado na origem; usado para descartar duplicatas, inclusive na sincronização da borda.
timestamp	Momento da **medição**, em UTC (ISO 8601). A plataforma ordena pelo horário da medição, não pela chegada: um dado que chega dois dias depois entra no lugar certo do histórico.
via	Opcional: nó de borda ou gateway pelo qual a mensagem passou (rastreabilidade).
local	Site, área e geometria GeoJSON, na ordem longitude, latitude. Se omitido, vale o local cadastrado do dispositivo.
medidas	Grandezas do catálogo, com unidades no código UCUM (ex.: Cel, %, ug/m3).

4.5 Convenção de tópicos MQTT

```
tópicos (AsyncAPI)
{inquilino}/{site}/{dominio}/{dispositivo}/telemetria   dispositivo -> plataforma
{inquilino}/{site}/{dominio}/{dispositivo}/evento       dispositivo -> plataforma
{inquilino}/{site}/{dominio}/{dispositivo}/estado       dispositivo -> plataforma (gêmeo)
{inquilino}/{site}/{dominio}/{dispositivo}/comando      plataforma  -> dispositivo

dominio: valor do catálogo do inquilino (configurável, sem lista fixa)
cidade:  semma/centro/ambiente/ar-centro-03/telemetria
campo:   coop-cerrado/fazenda-boa-vista/agricultura/solo-talhao07-02/telemetria
```

4.6 API REST (resumo)

Método e caminho	Finalidade
POST /api/v1/dispositivos	Cadastrar dispositivo e obter credenciais.
POST /api/v1/ingestao	Enviar uma mensagem por HTTP (para quem não usa MQTT).
POST /api/v1/ingestao/lote	Enviar um lote de mensagens (sincronização de nós de borda).
GET /api/v1/sites/{id}/areas	Listar áreas de um local, com geometria.
GET /api/v1/dispositivos/{id}/telemetria	Consultar histórico por grandeza e intervalo.
PUT /api/v1/dispositivos/{id}/gemeo/desejado	Definir o estado desejado (comando assíncrono).
POST /api/v1/catalogo/dominios	Criar um domínio novo no catálogo do inquilino.
POST /api/v1/regras	Criar regra de processamento (central ou de borda).
GET /ngsi-ld/v1/entities?type=AgriParcel	Consultar o contexto no padrão NGSI-LD.

SEÇÃO 5

Requisitos funcionais

Os requisitos de cada módulo, com prioridade e critério de aceitação verificável.

5.1 Cenários e catálogo (CEN)

ID	Requisito	Prioridade	Critério de aceitação
RF-CEN-01	Manter um catálogo de domínios configurável por inquilino.	Essencial	Gestor cria o domínio “pecuaria” pela API ou painel, sem alteração de código.
RF-CEN-02	Definir perfis de dispositivo: grandezas, unidades, faixa válida e intervalo esperado de envio.	Essencial	Medida fora da faixa do perfil é marcada como suspeita; perfil é reutilizável entre projetos.
RF-CEN-03	Organizar locais em hierarquia: inquilino → local → área → dispositivo.	Essencial	A mesma hierarquia representa um bairro com praças e uma fazenda com talhões.
RF-CEN-04	Registrar geometrias GeoJSON (ponto, polígono, linha) e permitir consultas espaciais.	Importante	Consulta “dispositivos dentro do talhão 07” retorna exatamente os sensores do polígono.
RF-CEN-05	Oferecer modelos de cenário (urbano e rural) que pré-carregam domínios, perfis e painéis.	Desejável	Aplicar o modelo rural a um inquilino novo cria seu catálogo inicial.
RF-CEN-06	Mapear os domínios para Smart Data Models (Smart Cities e AgriFood).	Importante	Entidades rurais e urbanas são expostas em NGSI-LD com os tipos padronizados.

5.2 Conectividade e ingestão (ING)



Figura 5.1 · Pipeline de ingestão no padrão Pipes and Filters. A etapa “Ordenar” posiciona cada medição pelo seu horário, inclusive as que chegam atrasadas de um nó de borda.

ID	Requisito	Prioridade	Critério de aceitação
RF-ING-01	Receber telemetria via MQTT 3.1.1 e 5.0 sobre TLS.	Essencial	Dispositivo autenticado publica no tópico canônico e a mensagem é persistida em até 2 s.
RF-ING-02	Receber telemetria via HTTPS (POST /api/v1/ingestao).	Essencial	Requisição válida retorna 202; inválida retorna 400 com a lista de erros.
RF-ING-03	Receber lotes de mensagens (POST /api/v1/ingestao/lote), com resultado por mensagem.	Essencial	Lote de 5.000 mensagens com 3 inválidas grava 4.997 e informa as 3 recusadas.
RF-ING-04	Aceitar dados atrasados, armazenando-os pelo horário da medição.	Essencial	Medição de 48 h atrás entra na posição correta do histórico e não gera alarme como se fosse atual.

ID	Requisito	Prioridade	Critério de aceitação
RF-ING-05	Integrar servidores de rede LoRaWAN (ChirpStack, The Things Stack), públicos ou privados.	Importante	Uplink LoRaWAN aparece como telemetria canônica do dispositivo.
RF-ING-06	Validar cada mensagem contra o JSON Schema da sua versão.	Essencial	Mensagem fora do esquema vai para a fila de mensagens inválidas, com o motivo.
RF-ING-07	Aceitar cargas em JSON e em CBOR, e lotes compactados.	Importante	A mesma mensagem nos dois formatos gera o mesmo registro.
RF-ING-08	Descartar mensagens duplicadas pelo id_mensagem.	Essencial	Mensagem reenviada pela borda com o mesmo id é armazenada uma única vez.
RF-ING-09	Receber telemetria via CoAP sobre DTLS.	Desejável	Cliente CoAP de teste publica com sucesso.
RF-ING-10	Limitar a taxa de mensagens por dispositivo e por inquilino.	Importante	Ao exceder o limite configurado, o excesso é recusado e registrado.

5.3 Borda e conectividade limitada (EDG)

ID	Requisito	Prioridade	Critério de aceitação
RF-EDG-01	Executar um nó de borda com ingestão local (MQTT e HTTP) e fila persistente.	Essencial	Com o link desligado por 72 h, nenhuma mensagem aceita pelo nó é perdida.
RF-EDG-02	Sincronizar com a instância central ao reconectar, em lotes, sem perdas nem duplicatas.	Essencial	Após a reconexão, cada mensagem chega uma única vez, ordenada pelo horário da medição.
RF-EDG-03	Executar regras locais no nó de borda, sem depender da instância central.	Importante	Regra de irrigação dispara com o link desligado; o evento é sincronizado depois.
RF-EDG-04	Receber da instância central configurações, perfis e regras quando conectado.	Importante	Regra alterada na central passa a valer no nó na próxima conexão.
RF-EDG-05	Priorizar o envio quando a banda é limitada: alarmes antes da telemetria rotineira.	Importante	Num link lento, um alarme gerado agora chega antes do lote atrasado de telemetria.
RF-EDG-06	Gerenciar remotamente os nós de borda: estado, versão, fila pendente e espaço livre.	Importante	Painel mostra há quanto tempo cada nó está sem sincronizar.
RF-EDG-07	Oferecer um painel local mínimo, acessível na rede da propriedade sem internet.	Desejável	Produtor vê as últimas leituras pelo celular na Wi-Fi da sede, sem internet.

5.4 Gestão de dispositivos (DIS)

ID	Requisito	Prioridade	Critério de aceitação
RF-DIS-01	Cadastrar dispositivos com perfil, domínio, local, área, fabricante, firmware e inquilino.	Essencial	Dispositivo cadastrado pela API e pelo painel aparece com todos os atributos.
RF-DIS-02	Emitir credenciais únicas por dispositivo (certificado X.509 ou token) e permitir revogá-las.	Essencial	Credencial revogada é recusada na conexão seguinte, na central e nos nós de borda.
RF-DIS-03	Provisionar dispositivos em lote (arquivo CSV ou API).	Importante	Importação de 1.000 dispositivos conclui com relatório de sucessos e falhas.
RF-DIS-04	Manter o gêmeo de cada dispositivo (estado desejado, relatado e diferença).	Importante	Alterar o desejado com o aparelho offline gera a diferença, entregue na reconexão.
RF-DIS-05	Enviar comandos aos dispositivos e registrar a resposta.	Importante	Comando chega ao tópico de comando e a resposta é associada a ele.
RF-DIS-06	Indicar online/offline usando o intervalo esperado do perfil.	Essencial	Sensor rural que envia de hora em hora não aparece offline após 5 minutos sem dados.
RF-DIS-07	Organizar dispositivos em grupos e etiquetas (tags).	Importante	Consulta por tag retorna apenas os dispositivos marcados.
RF-DIS-08	Atualizar firmware remotamente (OTA) em ondas, com grupo canário e parada por limite de falhas.	Desejável	Distribuição para quando a taxa de falhas do grupo passa do limite configurado.

5.5 Integração de projetos existentes (INT)

ID	Requisito	Prioridade	Critério de aceitação
RF-INT-01	Publicar o contrato canônico versionado (JSON Schema e AsyncAPI).	Essencial	Contrato acessível por URL pública, com histórico de versões.
RF-INT-02	Permitir a integração usando apenas protocolos padrão, sem biblioteca proprietária.	Essencial	Exemplos funcionais em três linguagens usando apenas bibliotecas genéricas de MQTT/HTTP.
RF-INT-03	Executar adaptadores de protocolo como componentes plugáveis em contêiner, em qualquer linguagem.	Essencial	Novo adaptador é implantado sem recompilar nem reiniciar o núcleo (central ou borda).
RF-INT-04	Conectar sistemas existentes por API REST (coleta agendada) e por webhooks (envio).	Importante	Conector traz os dados de um portal de telemetria de máquinas no formato canônico.
RF-INT-05	Expor os dados de contexto no padrão NGSI-LD.	Importante	Consulta NGSI-LD retorna entidades válidas (ex.: AgriParcel, WasteContainer).
RF-INT-06	Oferecer ambiente de testes (sandbox) e validador de mensagens para integradores.	Importante	Integrador valida uma mensagem sem afetar dados de produção.
RF-INT-07	Fornecer o kit do integrador: guia, exemplos e coleção de requisições.	Importante	Um integrador novo conecta um dispositivo de teste seguindo só o guia.

ID	Requisito	Prioridade	Critério de aceitação
RF-INT-08	Permitir mapeamento de campos (de-para) configurável, sem código, para JSON legado simples.	Desejável	Mapeamento criado no painel converte a mensagem legada para a canônica.

5.6 Processamento e regras (PRO)

ID	Requisito	Prioridade	Critério de aceitação
RF-PRO-01	Normalizar unidades e enriquecer mensagens com perfil, domínio, local e área.	Essencial	Mensagem armazenada contém inquilino, domínio, local e área do cadastro.
RF-PRO-02	Avaliar regras condição → ação: limite, ausência de dados, janela de tempo e área geográfica.	Essencial	Regra “temperatura do silo > 30 °C” dispara em até 3 s após a mensagem.
RF-PRO-03	Executar ações: gerar alarme, notificar, enviar comando, chamar webhook e publicar em tópico.	Essencial	Cada tipo de ação é demonstrado por um teste automatizado.
RF-PRO-04	Calcular agregações em janelas (média, mínimo e máximo por hora e por dia).	Importante	Consulta agregada retorna os mesmos valores calculados sobre os dados brutos.
RF-PRO-05	Gerenciar o ciclo de vida dos alarmes: ativo, reconhecido e resolvido, com severidade.	Essencial	Alarme gerado na borda durante a desconexão aparece com o horário em que ocorreu.
RF-PRO-06	Estender o processamento com funções externas (webhook) em qualquer linguagem.	Desejável	Função externa recebe a mensagem e devolve o resultado ao pipeline.

5.7 Dados (DAD)

ID	Requisito	Prioridade	Critério de aceitação
RF-DAD-01	Armazenar séries temporais com consulta por dispositivo, grandeza, área e intervalo.	Essencial	Consulta de 24 h de um dispositivo responde em até 1 s.
RF-DAD-02	Manter o último valor de cada grandeza de cada dispositivo.	Essencial	Último estado disponível sem consultar o histórico.
RF-DAD-03	Aplicar políticas de retenção e de redução de resolução (downsampling).	Importante	Dados brutos mais antigos que o prazo são substituídos por agregados.
RF-DAD-04	Exportar dados em CSV e JSON e publicar conjuntos de dados abertos.	Importante	Exportação respeita o filtro aplicado e o perfil de acesso do usuário.
RF-DAD-05	Anonimizar ou pseudonimizar dados pessoais antes de armazenar ou publicar.	Essencial	Nenhum dado pessoal identificável aparece na API pública.
RF-DAD-06	Registrar trilha de auditoria das operações administrativas.	Importante	Toda alteração de dispositivo, regra ou permissão tem autor, data e valor anterior.

5.8 Aplicação e serviços (APP)

ID	Requisito	Prioridade	Critério de aceitação
RF-APP-01	Disponibilizar API REST versionada e documentada em OpenAPI 3.	Essencial	Documentação interativa gerada a partir do contrato e testada no pipeline.
RF-APP-02	Permitir assinatura de dados em tempo real (WebSocket ou MQTT).	Importante	Aplicação assinante recebe a telemetria em até 2 s após a ingestão.
RF-APP-03	Enviar webhooks de saída para eventos e alarmes, com novas tentativas.	Importante	Falha no destino gera novas tentativas com espera crescente.
RF-APP-04	Oferecer painéis configuráveis: mapa com pontos e áreas, gráficos, indicadores e alarmes.	Importante	Gestor monta um painel com o mapa dos talhões ou dos bairros, sem programar.
RF-APP-05	Notificar por e-mail, aplicativo de mensagens e SMS.	Importante	Alarme crítico chega ao produtor por SMS mesmo sem internet móvel.
RF-APP-06	Ter interface web responsiva, utilizável em celular com conexão lenta.	Importante	Painel principal carrega em até 5 s numa conexão de 1 Mbit/s.
RF-APP-07	Padronizar paginação, filtros e ordenação em todas as listagens da API.	Essencial	Todas as listagens aceitam os mesmos parâmetros de paginação.

5.9 Segurança e acesso (SEG)

ID	Requisito	Prioridade	Critério de aceitação
RF-SEG-01	Autenticar usuários por OpenID Connect (OAuth 2.0), com segundo fator para administradores.	Essencial	Administrador sem segundo fator não acessa o painel.
RF-SEG-02	Controlar acesso por papéis: administrador, gestor, operador, integrador e leitor.	Essencial	Operador não consegue alterar regras nem dispositivos.
RF-SEG-03	Isolar os dados de cada inquilino (secretaria, cooperativa ou fazenda).	Essencial	Usuário de um inquilino não vê dados de outro, nem pela API.
RF-SEG-04	Restringir cada dispositivo aos próprios tópicos MQTT, na central e na borda.	Essencial	Publicação em tópico de outro dispositivo é recusada.
RF-SEG-05	Autenticar os nós de borda com credencial própria e cifrar a fila local.	Essencial	Disco retirado de um nó de borda não revela os dados guardados.
RF-SEG-06	Emitir chaves de API para integrações, com escopos e validade.	Importante	Chave expirada ou fora do escopo é recusada com 401 ou 403.

5.10 Observabilidade e administração (OBS)

ID	Requisito	Prioridade	Critério de aceitação
RF-OBS-01	Expor métricas da plataforma: mensagens por segundo, latência e erros por módulo.	Essencial	Métricas disponíveis em formato Prometheus.
RF-OBS-02	Gerar logs estruturados (JSON) com identificador de correlação.	Essencial	Uma mensagem pode ser rastreada em todos os módulos, da borda à central.
RF-OBS-03	Registrar rastreamento distribuído com OpenTelemetry.	Importante	Trace mostra o caminho da ingestão até a persistência.
RF-OBS-04	Exibir painel de saúde por integração e por nó de borda.	Importante	Gestor identifica qual integração ou qual fazenda está com falha.
RF-OBS-05	Gerar alertas operacionais da própria plataforma (fila crescendo, nó sem sincronizar).	Importante	Nó de borda sem sincronizar por mais que o limite gera alerta ao administrador.

SEÇÃO 6

Requisitos não funcionais

Qualidade, desempenho, segurança e operação em cenários conectados e desconectados, sempre com métrica.

ID	Atributo	Requisito e métrica
RNF-01	Desempenho	Sustentar 1.000 mensagens/s no MVP em um servidor central; chegar a 10.000 mensagens/s adicionando réplicas.
RNF-02	Latência	Com conexão: p95 da ingestão até a persistência ≤ 2 s e até o disparo de uma regra ≤ 3 s. Regras de borda: ≤ 3 s, independentemente do link.
RNF-03	Disponibilidade	99,5% ao mês para ingestão e API da instância central.
RNF-04	Tolerância a desconexão	Nó de borda opera ao menos 72 h sem link, sem perda de dados.
RNF-05	Banda limitada	Sincronização da borda funciona em links a partir de 64 kbit/s, com cargas compactadas.
RNF-06	Hardware de borda	Nó de borda executa em equipamento com 4 núcleos ARM e 4 GB de RAM (ex.: Raspberry Pi 5).
RNF-07	Robustez a falta de energia	Queda de energia não corrompe a fila nem a configuração do nó de borda (escrita transacional).
RNF-08	Neutralidade de cenário	Um domínio, perfil ou modelo de cenário novo é criado só por configuração, sem alteração de código.
RNF-09	Escalabilidade	Serviços centrais sem estado, escaláveis horizontalmente.
RNF-10	Durabilidade	Nenhuma mensagem confirmada ao emissor (ACK) pode ser perdida, na central ou na borda.
RNF-11	Segurança	TLS 1.2 ou superior em toda interface externa; dados e segredos criptografados em repouso.
RNF-12	Privacidade	Conformidade com a LGPD: minimização, finalidade e registro das operações de tratamento.
RNF-13	Independência de linguagem	100% das interfaces descritas por contratos abertos (OpenAPI, AsyncAPI, JSON Schema, GeoJSON).
RNF-14	Portabilidade	Executar em qualquer nuvem ou localmente, em contêineres OCI: Docker Compose (MVP e borda) ou Kubernetes.
RNF-15	Manutenibilidade	Cobertura de testes automatizados ≥ 70% no núcleo; análise estática sem falhas críticas no pipeline.
RNF-16	Usabilidade e acessibilidade	Interface em português, responsiva; conformidade com WCAG 2.1 nível AA.
RNF-17	Compatibilidade	APIs versionadas; uma versão descontinuada continua funcionando por 12 meses.
RNF-18	Recuperação	Backup diário da central; RPO ≤ 24 h e RTO ≤ 4 h.

SEÇÃO 7

Padrões de projeto e boas práticas

As decisões de projeto que tornam a plataforma desacoplada, extensível, neutra quanto ao cenário e fácil de manter.

7.1 Arquitetura de cada serviço: hexagonal



Figura 7.1 · Arquitetura hexagonal (portas e adaptadores) aplicada a cada serviço: o domínio não conhece protocolo, banco, fornecedor nem cenário.

Cada serviço separa o **domínio** (regras de negócio) das **portas** (interfaces que expõe ou de que precisa) e dos **adaptadores** (implementações concretas: MQTT, REST, lotes da borda, banco). Trocar o banco ou incluir um novo protocolo significa escrever um adaptador novo, sem tocar no domínio. O mesmo domínio roda na instância central e no nó de borda, com adaptadores diferentes.

7.2 Padrões adotados

Padrão	Onde se aplica	Benefício
Arquitetura em camadas	Organização geral da plataforma	Separação de responsabilidades entre camadas.
Portas e adaptadores (hexagonal)	Cada serviço, central e de borda	Domínio independente de protocolo, banco e fornecedor.
Type Object	Domínios e perfis de dispositivo	Novos cenários e tipos de dispositivo sem criar classes nem alterar código (RNF-08).
Adapter (GoF)	Adaptadores de protocolo (nível 2)	Integra formatos legados sem alterar o núcleo.
Camada anticorrupção (DDD)	Conectores de sistemas existentes (nível 3)	O modelo legado não contamina o modelo canônico.
Modelo canônico de dados (EIP)	Contrato de mensagens	N formatos de entrada viram um só formato interno.
Store-and-forward / entrega garantida (EIP)	Nó de borda	Nada se perde durante a desconexão.
Publicação/assinatura (EIP)	Barramento interno	Produtores e consumidores desacoplados.

Padrão	Onde se aplica	Benefício
Pipes and Filters (EIP)	Pipeline de ingestão	Etapas independentes, testáveis e reordenáveis.
Dead Letter Channel (EIP)	Mensagens inválidas	Nenhuma mensagem se perde em silêncio.
Idempotent Receiver (EIP)	Ingestão e sincronização da borda	Reenvios não geram registros repetidos.
Strategy (GoF)	Ações do motor de regras	Novas ações sem alterar o motor.
Observer (GoF)	Alarmes e notificações	Vários interessados reagem ao mesmo evento.
Repository	Acesso a dados	Troca de banco sem afetar o domínio (ex.: SQLite na borda, PostgreSQL na central).
Device Shadow / gêmeo	Gestão de dispositivos	Comandos assíncronos para aparelhos offline.
CQRS	Ingestão (escrita) × consultas (leitura)	Escala e otimiza cada lado separadamente.
API Gateway	Borda da camada de aplicação	Autenticação, limites e versionamento num só ponto.
Circuit Breaker e Retry com backoff	Conectores, webhooks e sincronização	Falha externa não derruba a plataforma.
Plugin	Adaptadores e conectores	Extensão sem recompilar o núcleo (RF-INT-03).

7.3 Boas práticas de programação e engenharia

Prática	Como se aplica na plataforma
Princípios SOLID	Responsabilidade única; dependência de interfaces (portas), não de implementações; aberto para extensão (novos adaptadores) e fechado para modificação (núcleo).
Configuração em vez de código	Domínios, perfis, locais e regras são dados configuráveis, nunca constantes no código.
Código limpo	Nomes expressivos, funções curtas, sem duplicação; revisão de código obrigatória.
Contrato primeiro	OpenAPI, AsyncAPI e JSON Schema são escritos antes do código e validados no pipeline.
Versionamento semântico	Contratos e serviços seguem MAJOR.MINOR.PATCH; quebra de contrato só em versão MAJOR.
Testes automatizados	Unidade, integração, contrato, carga e **testes de desconexão** (link cortado por horas, queda de energia).
Integração e entrega contínuas	Build, testes, análise estática e varredura de vulnerabilidades a cada alteração; a mesma imagem serve à central e à borda.
Infraestrutura como código	Ambientes descritos em arquivos versionados (Compose, Helm, Terraform).
Twelve-Factor App	Configuração externa, serviços sem estado, logs como fluxo de eventos.
Segurança desde o projeto	Menor privilégio, segredos fora do código, OWASP API Security e OWASP IoT Top 10.
Registro de decisões (ADR)	Cada decisão de arquitetura documentada com contexto, alternativas e consequências.

SEÇÃO 8

Casos de uso

Fluxos que validam a arquitetura de ponta a ponta, na cidade, no campo e na criação de um cenário novo.

UC-01 · Integrar uma fazenda com link via satélite

Ator principal: Cooperativa e integrador · **Requisitos:** RF-CEN-03, RF-CEN-04, RF-EDG-01 a 03, RF-ING-03, RF-ING-04

1. A cooperativa cadastra o local “Fazenda Boa Vista” e desenha no mapa os talhões como áreas (RF-CEN-03, RF-CEN-04).
2. Um nó de borda é instalado na sede, com um gateway LoRaWAN; os sensores de solo publicam nele (RF-EDG-01).
3. À noite, o link via satélite cai por 10 horas. O nó guarda as leituras e a regra local liga a irrigação do talhão 07 quando a umidade cai (RF-EDG-03).
4. Ao reconectar, o nó envia os dados em lotes (RF-EDG-02, RF-ING-03). A central os registra na hora da medição (RF-ING-04) e mostra o acionamento da irrigação como evento histórico.

UC-02 · Alarme de temperatura em silo

Ator principal: Produtor · **Requisitos:** RF-CEN-02, RF-PRO-02, RF-PRO-05, RF-APP-05

1. O perfil “termometria de silo” define a faixa válida e o envio a cada 15 minutos (RF-CEN-02).
2. A regra “temperatura > 30 °C por 30 minutos → alarme crítico + SMS” é criada (RF-PRO-02).
3. Ao ser atendida, a regra gera o alarme e o produtor recebe o SMS mesmo sem internet móvel (RF-APP-05).

UC-03 · Integrar um projeto urbano de qualidade do ar (nível 2)

Ator principal: Integrador · **Requisitos:** RF-INT-03, RF-INT-05, RF-INT-06, RF-ING-06

1. O gestor cria o inquilino “Meio Ambiente” e o integrador recebe uma chave de API de sandbox.
2. As estações publicam um JSON próprio; o integrador escreve um adaptador, em qualquer linguagem, e o valida no sandbox (RF-INT-06).
3. O adaptador é implantado como contêiner plugável (RF-INT-03) e as medições aparecem no histórico e na API NGSI-LD.

UC-04 · Alarme de lixeira cheia e rota de coleta

Ator principal: Gestor e operador · **Requisitos:** RF-PRO-02, RF-PRO-03, RF-PRO-05, RF-APP-03

1. A regra “nível > 80% por 10 minutos → alarme + webhook” é criada (RF-PRO-02).
2. A plataforma gera o alarme e chama o webhook do sistema de rotas (RF-APP-03), que inclui a lixeira na rota.
3. O operador reconhece o alarme; após a coleta, o nível cai e o alarme é resolvido.

UC-05 · Dimerizar a iluminação de uma praça

Ator principal: Gestor · **Requisitos:** RF-DIS-04, RF-DIS-05, RF-DIS-07

1. O gestor define no gêmeo do grupo de luminárias o brilho desejado de 40% após a meia-noite (RF-DIS-04).
2. Luminárias online recebem o comando; as offline recebem a diferença quando reconectam.
3. Cada luminária relata o novo estado e a diferença desaparece.

UC-06 · Criar um cenário novo sem programar: reservatórios de água

Ator principal: Gestor · **Requisitos:** RF-CEN-01, RF-CEN-02, RNF-08

1. O gestor cria o domínio “recursos_hidricos” no catálogo (RF-CEN-01).
2. Cria o perfil “nível de reservatório”: grandeza nível (m), faixa 0 a 10 m, envio a cada 30 minutos (RF-CEN-02).
3. O integrador conecta os sensores pelo nível 1, e o domínio novo já tem histórico, regras, painéis e API, sem nenhuma alteração de código (RNF-08).

SEÇÃO 9

Tecnologias de referência e roteiro

Sugestões de componentes abertos para a central e para a borda, e as fases de entrega.

9.1 Tecnologias de referência

As tecnologias abaixo são **sugestões**, todas de código aberto. Por causa da arquitetura hexagonal, qualquer uma pode ser substituída por um equivalente. Os serviços próprios da plataforma podem ser escritos na linguagem que a equipe dominar, já que se comunicam apenas por contratos.

Camada	Opções de referência
Conectividade e ingestão	EMQX ou Eclipse Mosquitto (MQTT); Eclipse Californium (CoAP); ChirpStack (LoRaWAN)
Barramento interno	Apache Kafka, NATS ou RabbitMQ
Gestão e regras	Serviços próprios; Node-RED para protótipos de regras
Dados	PostgreSQL + TimescaleDB (séries) e PostGIS (geometrias); Redis (último estado); Orion-LD (NGSI-LD)
Aplicação	API Gateway (Kong ou Traefik); Grafana (painéis); Swagger UI (documentação)
Segurança	Keycloak (OpenID Connect, papéis); Vault (segredos); CA própria para certificados
Observabilidade	OpenTelemetry, Prometheus, Grafana, Loki
Nó de borda	Mosquitto, ChirpStack local, SQLite como fila persistente, motor de regras local; Raspberry Pi 5 ou mini PC industrial
Conectividade rural	LoRaWAN privada, celular rural, satélite (baixa órbita ou geoestacionário)
Implantação	Docker Compose (MVP e borda), Kubernetes com Helm (escala), GitHub Actions ou GitLab CI

9.2 Roteiro de entrega

Fase	Entregas	Requisitos
1 · MVP	Catálogo, perfis e locais; ingestão MQTT, HTTP e lotes com validação, deduplicação e dados atrasados; nó de borda com fila e sincronização; cadastro e credenciais; séries e último estado; API; painel básico; papéis e inquilinos; métricas e logs.	Todos os “Essencial”
2 · Integração	Regras de borda, geometrias e consultas espaciais, adaptadores plugáveis, conectores, LoRaWAN, NGSI-LD, gêmeo e comandos, agregações, notificações e SMS, sandbox e kit do integrador.	“Importante”
3 · Evolução	Modelos de cenário prontos, painel local da borda, CoAP, OTA com grupo canário, funções externas e mapeamento sem código.	“Desejável”

9.3 Critérios de aceite do MVP

- Um projeto urbano e um projeto rural integrados; o rural, com nó de borda.
- Teste de desconexão: link da borda cortado por 72 h sem perda nem duplicação de dados após a reconexão.
- Um domínio novo criado só por configuração (RNF-08).
- Integração demonstrada em pelo menos três linguagens de programação diferentes.
- Teste de carga comprovando o RNF-01 e o RNF-02 em ambiente de homologação.

SEÇÃO 10

Glossário e referências

Termos usados no documento e as fontes de referência.

10.1 Glossário

Termo	Definição
ACK	Confirmação de recebimento enviada ao emissor da mensagem.
Área	Subdivisão de um local com geometria própria: praça, talhão, pasto, silo.
AsyncAPI	Especificação aberta para descrever interfaces assíncronas, como tópicos MQTT.
Camada anticorrupção	Componente que traduz o modelo de um sistema externo para o modelo interno.
Dado atrasado	Medição que chega à central depois de outras mais recentes; é registrada pelo horário da medição.
DLQ	Fila de mensagens inválidas, onde ficam as mensagens rejeitadas para análise.
GeoJSON	Formato aberto (RFC 7946) para pontos, linhas e polígonos geográficos.
Gêmeo do dispositivo	Documento com o estado desejado e o relatado de um dispositivo.
Inquilino	Unidade isolada de dados e usuários: secretaria, cooperativa ou fazenda.
Local (site)	Lugar físico de um inquilino: bairro, campus, fazenda.
Mensagem canônica	Formato único e versionado em que todas as mensagens circulam na plataforma.
MVP	Produto mínimo viável: a primeira versão utilizável.
NGSI-LD	Padrão ETSI para dados de contexto, usado pelo FIWARE.
Nó de borda	Versão reduzida da plataforma instalada no local, que funciona desconectada e sincroniza depois.
Perfil de dispositivo	Definição reutilizável de um tipo de dispositivo: grandezas, unidades, faixas e intervalo.
Store-and-forward	Guardar localmente e reenviar quando houver conexão.
UCUM	Código unificado de unidades de medida (ex.: Cel, %, ug/m3).

Referências

- ISO/IEC/IEEE 29148:2018. *Systems and software engineering — Life cycle processes — Requirements engineering*.
- ISO/IEC 30141. *Internet of Things (IoT) — Reference architecture*.
- ETSI GS CIM 009. *Context Information Management (CIM); NGSI-LD API*.
- FIWARE FOUNDATION. *Smart Data Models* (domínios Smart Cities e Smart AgriFood). Disponível em: smartdatamodels.org.
- IETF. RFC 7946: *The GeoJSON Format*, 2016. *The Unified Code for Units of Measure (UCUM)*.
- OASIS. *MQTT Version 5.0*, 2019. OPENAPI INITIATIVE. *OpenAPI Specification*. ASYNCAPI INITIATIVE. *AsyncAPI Specification*.
- GAMMA, E. et al. *Padrões de Projeto: soluções reutilizáveis de software orientado a objetos*. Bookman, 2000.
- JOHNSON, R.; WOOLF, B. Type Object. In: *Pattern Languages of Program Design 3*. Addison-Wesley, 1997.
- HOHPE, G.; WOOLF, B. *Enterprise Integration Patterns*. Addison-Wesley, 2003.
- EVANS, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- COCKBURN, A. *Hexagonal Architecture* (Ports and Adapters), 2005.

- MARTIN, R. C. *Arquitetura Limpa*. Alta Books, 2019.
- WIGGINS, A. *The Twelve-Factor App*. Disponível em: 12factor.net.
- OWASP. *IoT Top 10* e *API Security Top 10*.
- MASCHIETTO, L. G. et al. *Arquitetura e Infraestrutura de IoT*. Porto Alegre: SAGAH, 2021.
- Documentação oficial: AWS IoT e IoT Greengrass, Microsoft Azure IoT e IoT Edge, ThingsBoard e ThingsBoard Edge, FIWARE.
- BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais).